Reliability-Aware Self-Healing Data Pipelines: Automated Failure Detection, Root-Cause Classification, and Policy-Constrained Recovery

Baharath Bathula
Independent Researcher
Dallas, Texas, USA
Email: baharath.bathula@outlook.com

***Abstract*—Reliable data pipelines support analytics, machine learning, compliance reporting, and operational decision-making, but failures such as schema drift, data-quality degradation, freshness violations, transformation defects, and corrupted outputs often require more than task-level retries or alerting. Existing systems provide workflow orchestration, data-quality validation, observability, incident response, and policy enforcement, but these capabilities are commonly evaluated as separate functions rather than as an end-to-end recovery lifecycle. This paper presents a reliability-aware self-healing control plane for data pipelines that separates failure detection, root-cause classification, policy-constrained remediation, execution, post-recovery validation, rollback or escalation, and incident evidence preservation. A reproducible synthetic experiment with 780 trials evaluates the framework across injected failures, healthy controls, and boundary controls. Under the current controlled experiment configuration, the framework achieved 100% failure detection accuracy and 100% root-cause classification accuracy, while verified recovery was achieved in 52.17% of injected failure trials. The mean self-healing cycle runtime was 12.609 ms, with a median of 10.609 ms and p95 of 26.666 ms. The results show that self-healing data pipelines should be evaluated using separate metrics for detection, diagnosis, and verified recovery because accurate detection does not guarantee safe or validated remediation.**

***Index Terms*—self-healing data pipelines, data reliability, data observability, root-cause classification, policy-constrained automation, failure injection, verified recovery**

I. Introduction

Enterprise data platforms increasingly depend on pipelines that ingest, transform, validate, and deliver data across distributed systems. These pipelines support analytics, machine learning, regulatory reporting, customer-facing applications, and operational decision-making. Prior work has shown that data pipelines are integral to modern data-driven systems but remain vulnerable to quality, processing, and maintenance problems, including data-related defects, pipeline complexity, and hidden technical debt in production systems [1]–[3]. As organizations increase the number of data sources, transformation layers, quality checks, and downstream consumers, pipeline reliability becomes harder to maintain through manual monitoring and incident response alone.

Data-pipeline failures can arise from schema drift, missing-value spikes, duplicate records, freshness violations, unexpected volume changes, transformation defects, source-system failures, corrupted outputs, and unknown failure modes. These failures may not always appear as simple task crashes. A pipeline can complete successfully while producing stale, incomplete, duplicated, or semantically invalid data. As a result, pipeline reliability requires more than workflow execution status. It requires detection, diagnosis, recovery decisioning, validation, and evidence preservation.

Existing data-engineering systems address important parts of this problem. Workflow orchestration tools schedule and execute pipeline tasks, data-quality frameworks validate assumptions about data, observability tools expose anomalies and operational signals, incident-management workflows coordinate response, and policy-as-code systems provide guardrails for automated decisions [4]–[8]. Recent work on recurring data-pipeline validation and comparative evaluations of data-quality tools further reinforces the importance of automated monitoring and validation for production-style pipelines [9], [10]. However, these capabilities are often evaluated separately. Detection accuracy does not prove recovery safety. A successful retry does not prove that the underlying data issue was resolved. A remediation action does not prove that the recovered pipeline state is valid.

This paper presents a policy-constrained self-healing control plane for data-pipeline failures. The framework evaluates self-healing as a staged lifecycle from failure detection to policy-approved and post-validated recovery. This design separates observability, diagnosis, and recovery outcomes so that the system is not rewarded for unsafe or unverifiable automation.

The key design principle is that automated recovery must be constrained by policy and validation. In production data environments, unrestricted remediation may create more risk than the original failure. For example, automatically dropping columns, rewriting outputs, backfilling stale data, or suppressing failed quality checks may violate governance rules or corrupt downstream data products. Therefore, this framework counts recovery as successful only when the remediation is approved, executed, and verified. Failures that are detected and classified but cannot be safely repaired are escalated rather than counted as successful self-healing.

The experimental evaluation uses a reproducible synthetic data-pipeline environment with controlled failure injection. The current experiment includes 780 total trials: 690 injected

failure trials, 60 healthy-control trials, and 30 boundary-control trials. Under the current synthetic experiment configuration, the framework achieved 100% failure detection accuracy and 100% root-cause classification accuracy. Verified recovery was achieved in 52.17% of injected failure trials. This separation between detection/classification performance and verified recovery performance is central to the paper's argument: self-healing data pipelines should be evaluated across multiple reliability stages, not only by whether a failure was detected or an automated action was attempted.

This paper makes the following contributions:

1) A staged self-healing control-plane architecture for data pipelines that separates failure detection, root-cause classification, remediation selection, policy approval, execution, validation, rollback, escalation, and evidence preservation.
2) A reproducible controlled failure-injection experiment with injected failure trials, healthy controls, and boundary controls.
3) A policy-constrained recovery model that prevents unsafe automation by requiring recovery actions to satisfy policy approval and post-recovery validation.
4) Separate reliability metrics for detection, diagnosis, and recovery, avoiding the misleading assumption that detection success implies recovery success.
5) A reproducibility package that preserves trial-level outputs, derived summaries, figure-generation scripts, and documentation for independent inspection and extension.

The remainder of this paper is organized as follows. Section 2 reviews related work in workflow orchestration, data-quality validation, observability, AIOps, autonomic computing, and policy-as-code. Section 3 describes the proposed self-healing control-plane architecture and experimental methodology. Section 4 presents experimental results across detection, classification, recovery, and runtime behavior. Section 5 discusses implications of the results. Section 6 describes threats to validity. Section 7 concludes with future research directions for policy-constrained self-healing data pipelines.

II. Related Work

Modern data platforms rely on a combination of workflow orchestration, data-quality validation, monitoring, incident management, and governance controls. Each area contributes to pipeline reliability, but existing approaches often treat detection, diagnosis, remediation, and validation as separate operational concerns. This paper builds on these areas while focusing on a narrower problem: how to evaluate policy-constrained self-healing for data-pipeline failures.

A. Workflow Orchestration

Workflow orchestration systems provide task scheduling, dependency management, retries, operational metadata, and execution control. Apache Airflow, for example, models workflows as Dags containing schedules, tasks, dependencies, callbacks, and operational parameters [4]. These capabilities are essential for data-pipeline execution because they define when tasks run, how tasks depend on one another, and how workflow instances are scheduled.

However, orchestration is not equivalent to self-healing. Airflow documentation explicitly frames the Dag as being concerned with execution order and task relationships rather than the internal semantics of each task [4]. Empirical work on Airflow-related developer challenges also shows that workflow definition and execution remain practical sources of difficulty for developers, especially in distributed execution environments [11]. A workflow engine can rerun a failed task or expose operational state, but it does not inherently determine whether a data-specific recovery action is safe, policy-approved, or validated after execution. This paper therefore treats orchestration as the execution substrate and evaluates self-healing as a separate reliability-control process.

B. Data-Quality Validation and Data Testing

Data-quality validation frameworks improve pipeline reliability by formalizing assumptions about data. Great Expectations provides expectations that define whether data conforms to expected conditions [5]. dbt data tests provide assertions over models and sources, returning records that fail specified assumptions [6]. These tools are directly relevant to self-healing because they expose structured validation signals that can help identify invalid pipeline states.

However, validation is not recovery. Data-quality validation can reveal that a pipeline output is stale, malformed, incomplete, duplicated, or inconsistent, but it does not by itself determine the root cause, select a safe remediation, evaluate policy constraints, execute recovery, verify post-recovery health, and preserve incident evidence. Research on data-pipeline quality reinforces this distinction by showing that data pipelines are affected by many quality factors and root causes, including data type issues, cleaning-stage failures, integration issues, and compatibility problems [1]. Related work on hidden technical debt also shows that data dependencies and pipeline jungles can create long-term maintenance risks in ML and data systems [2]. This paper uses data-quality validation as a signal source, but evaluates a broader recovery lifecycle.

C. Data Observability and Monitoring

Monitoring and observability systems collect operational signals about system state and pipeline behavior. In data systems, relevant signals include freshness, volume, schema stability, distribution drift, null-rate changes, duplicate rates, failed transformations, and downstream validation results. These signals help operators understand what is broken and why a pipeline may no longer be trustworthy. Site reliability engineering practice similarly emphasizes monitoring as a mechanism for understanding system behavior and generating actionable operational signals [8].

Recent research on recurring data pipelines has also highlighted the operational cost of manually monitoring and resolving data-quality issues at scale. Auto-Validate-by-History, for example, targets recurring production pipelines by using

historical execution statistics to detect data-quality issues and reports evaluation over 2000 production pipelines [9]. Broader tool evaluations also show that the data-quality tooling landscape includes both open-source and proprietary systems with different measurement capabilities and operational tradeoffs [10]. These works strengthen the motivation for automated monitoring, but they do not remove the need to evaluate what happens after detection.

The limitation is that observability commonly improves visibility and alerting but does not guarantee controlled recovery. A data observability system may identify that a table is stale or that a schema changed unexpectedly, but recovery still often depends on human operators, manually encoded runbooks, or ad hoc retries. Monitoring also does not determine whether an automated remediation is governance-safe or whether the recovered pipeline state is valid after repair. This paper therefore treats observability as one stage within a larger self-healing lifecycle rather than as the full solution.

D. AIOps and Root-Cause Analysis

AIOps research addresses anomaly detection, event correlation, root-cause analysis, incident management, and automated operations in complex IT systems. A systematic mapping study of AIOps reports substantial attention to failure-related tasks, including anomaly detection and root-cause analysis [12]. Other work has explored mining root-cause knowledge from cloud-service incident investigations, showing how incident evidence can be converted into reusable operational knowledge [13].

This literature is important because it motivates automated diagnosis under operational complexity. However, much AIOps research focuses on infrastructure, services, logs, microservices, or cloud operations rather than data-pipeline-specific failure modes. Even when AIOps approaches support root-cause analysis, they do not necessarily implement policy-constrained remediation for data pipelines or evaluate recovery success using post-recovery validation. This paper contributes to the data-engineering reliability setting by connecting root-cause classification to governed remediation outcomes.

E. Autonomic Computing and Self-Healing Systems

Autonomic computing introduced the vision of systems that manage themselves through self-configuration, self-healing, self-optimization, and self-protection [14]. This vision is directly relevant to data-pipeline reliability because pipeline failures increasingly exceed the capacity of manual operators to inspect, diagnose, and repair at scale. The autonomic-computing framing also emphasizes high-level goals and policy-guided self-management, which aligns with the need for governed recovery rather than unrestricted automation.

The gap is that autonomic computing is a broad systems paradigm. It does not specify a reproducible evaluation framework for data-pipeline failure detection, root-cause classification, policy approval, remediation execution, and post-recovery validation. This paper operationalizes the self-healing concept for data pipelines and evaluates each stage separately.

F. Policy-as-Code and Governance

Policy-as-code systems decouple policy decision-making from application logic and enforcement points. Open Policy Agent provides a policy engine and documentation for offloading policy decisions from software systems [7]. This architecture is relevant to self-healing pipelines because automated remediation must not be allowed to perform unsafe changes simply because a failure was detected.

However, policy engines are decision components, not complete self-healing systems. They can approve, deny, or structure decisions, but they do not generate telemetry, diagnose pipeline failures, execute repairs, validate recovered state, or preserve incident evidence. This paper uses policy-as-code as part of a larger reliability-control architecture in which policy decisions constrain recovery execution and verified recovery metrics.

G. Research Gap

Existing literature and systems provide strong building blocks for pipeline reliability: workflow orchestration executes tasks, data-quality frameworks validate expectations, observability platforms detect anomalies, AIOps methods support diagnosis, autonomic computing motivates self-management, and policy-as-code provides decision guardrails. The unresolved gap is the lack of an end-to-end, reproducible evaluation framework for data pipelines that measures the full lifecycle from injected failure to detection, classification, policy-constrained remediation, validation, rollback or escalation, and incident evidence.

This gap matters because real enterprise data pipelines cannot be safely repaired by unrestricted automation. A failure may be detected correctly and classified accurately while still requiring human escalation because the remediation is unsafe, unavailable, unverifiable, or governance-sensitive. Therefore, a credible self-healing system must distinguish between detection success, classification success, attempted remediation, policy-approved remediation, executed remediation, and verified recovery.

III. Methodology and Experimental Design

A. Overview

This study evaluates a policy-constrained self-healing control plane for data-pipeline failures. The framework is designed to model self-healing as a sequence of measurable reliability stages rather than as a single automated repair action. Each trial passes through failure injection or control setup, telemetry collection, failure detection, root-cause classification, remediation selection, policy evaluation, remediation execution, post-recovery validation, and evidence preservation.

The experimental goal is to determine whether the proposed framework can detect injected data-pipeline failures, classify their likely root causes, and safely recover when remediation is available, policy-approved, executable, and verifiable. The experiment therefore separates detection performance, diagnostic performance, and recovery performance. This separation is necessary because a pipeline failure can be detected and

TABLE I
SELF-HEALING CONTROL-PLANE STAGES.

Stage	Input	Output	Metric or evidence
Pipeline execution	Synthetic workload configuration	Pipeline output and execution state	Trial record
Failure injection	Scenario and severity configuration	Injected failure or control condition	Trial metadata
Telemetry collection	Pipeline outputs and execution state	Data-quality, structural, freshness, and runtime signals	Raw result fields
Failure detection	Telemetry signals	Failure or non-failure decision	Detection accuracy
Root-cause classification	Detected failure	Predicted root-cause label	Classification accuracy
Remediation selection	Predicted root cause	Candidate remediation action	Remediation action field
Policy evaluation	Candidate remediation	Approved, rejected, or human-approval-required decision	Policy/recovery status fields
Recovery execution	Approved remediation	Execution result	Remediation executed field
Post-recovery validation	Recovered pipeline output	Validated or non-validated recovery outcome	Recovery verified field
Rollback, escalation, and evidence	Trial outcome	Incident record and preserved evidence	Raw and derived artifacts

classified correctly while still requiring escalation if automated recovery is unsafe or unverifiable.

The current evaluation uses a reproducible synthetic data-pipeline environment with controlled failure injection. The experiment includes injected failure trials, healthy-control trials, and boundary-control trials. Injected failure trials test the system under known failure conditions. Healthy controls test whether the system avoids reporting failures when no failure is present. Boundary controls test whether the system behaves conservatively near decision thresholds rather than over-triggering remediation.

B. System Architecture

The proposed system is organized as a reliability-control plane layered above a data-pipeline execution environment. The control plane contains the following stages.

This architecture intentionally prevents the system from treating all detected failures as candidates for automatic repair. A recovery action is considered successful only when it passes policy evaluation and post-recovery validation.

C. Failure Taxonomy

The experiment uses a predefined data-pipeline failure taxonomy. The taxonomy is based on common reliability problems in data pipelines where task-level success does not necessarily imply data correctness.

The taxonomy supports deterministic evaluation because each injected trial has a known expected failure category. However, this also limits external validity. Real production failures may be overlapping, delayed, partially observable, or semantically ambiguous.

TABLE II
FAILURE TAXONOMY USED IN THE CONTROLLED EXPERIMENT.

Failure category	Description	Expected system behavior
Schema drift	Input schema differs from expected structure	Detect structural mismatch, classify schema drift, remediate only if policy permits
Missing-value spike	Null or missing values exceed expected threshold	Detect quality degradation, classify missing-value issue, validate recovered output
Duplicate-record generation	Duplicate records appear above expected tolerance	Detect duplicate spike, classify duplicate failure, remediate or escalate
Data-freshness violation	Data is stale or late relative to expected timing	Detect freshness issue, classify freshness violation, recover or escalate
Unexpected volume change	Row count or volume deviates from expected range	Detect abnormal volume, classify volume anomaly
Transformation-logic failure	Transformation produces invalid or inconsistent outputs	Detect downstream invalid state, classify transformation failure
Source-system failure	Upstream source is unavailable or returns invalid data	Detect source-related failure, classify source-system issue, escalate if unsafe
Partial or corrupted output	Output is incomplete or malformed	Detect output corruption, classify corruption or partial output
Unknown failure	Failure does not map cleanly to supported classes	Detect abnormal state where possible, avoid unsafe recovery, escalate
Healthy control	No failure is injected	Avoid false-positive detection
Boundary control	Near-threshold or ambiguous condition	Avoid aggressive remediation unless confidence and policy allow it

TABLE III
EVALUATION METRICS USED IN THE EXPERIMENT.

Trial type	Count
Injected failure trials	690
Healthy-control trials	60
Boundary-control trials	30
Total trials	780

D. Trial Design

Each experiment trial represents one pipeline execution under a defined scenario. A scenario is defined by the experiment domain, injected failure type, severity or control condition, and expected outcome. The trial-level record preserves the injected condition, observed detection outcome, predicted root cause, recovery decision, policy decision, validation result, and runtime information.

The current experiment contains 780 total trials.

Injected failure trials are used to evaluate detection, root-cause classification, and verified recovery. Healthy controls are used to evaluate whether the framework avoids false alarms when no failure is present. Boundary controls are used to evaluate whether the framework behaves conservatively under near-threshold or ambiguous conditions.

The experiment domains represented in the preserved trial

records are `artifact` and `dataframe`. These domains represent evaluated implementation contexts, not alert-only or static-rule baseline comparisons. Although the broader research objective supports comparison with alert-only and static-rule approaches, the current manuscript reports only experiment domains represented in the preserved trial records. Full baseline comparison remains future work unless comparable baseline trials are added under identical failure scenarios and metrics.

E. Experimental Procedure

Each trial follows a consistent procedure. The trial is first assigned an experiment domain, failure type or control condition, severity level, and expected outcome. A synthetic data-pipeline workload is then executed using the trial configuration. For injected failure trials, a controlled fault is introduced. For healthy-control trials, no fault is injected. For boundary-control trials, the input is configured near a decision threshold.

The framework records execution state, data-quality signals, structural signals, volume and freshness indicators, and derived validation outputs. The detection layer determines whether the trial represents a failure condition. If a failure is detected, the classifier assigns the expected root-cause category. The system then checks whether a remediation action exists for the classified failure type. If a candidate remediation exists, the policy layer approves or rejects the proposed action based on predefined safety and governance constraints.

If the remediation is available and policy-approved, the recovery-execution layer performs the remediation. The system then validates whether the post-recovery output satisfies expected correctness and health conditions. A recovery is counted as verified only if the remediation was approved, executed, and validated. The trial record is preserved for downstream analysis, figure generation, and reproducibility.

F. Metrics

The experiment reports separate metrics for detection, classification, recovery, and runtime. The metrics are separated because they answer different reliability questions.

Detection accuracy measures whether the framework correctly identifies the presence or absence of a failure condition.

“‘text Detection Accuracy = Correct Detection Decisions / Total Detection Decisions “‘

For injected failure trials, detection is successful when the system marks the trial as failed. For healthy-control trials, detection is successful when the system does not mark the trial as failed. For boundary controls, detection behavior is interpreted based on the expected boundary outcome defined by the trial configuration.

Root-cause classification accuracy measures whether the predicted root-cause category matches the injected failure category.

“‘text Classification Accuracy = Correct Root-Cause Predictions / Classified Injected Failure Trials “‘

Verified recovery rate measures the percentage of injected failure trials that are safely recovered.

“‘text Verified Recovery Rate = Verified Recoveries / Injected Failure Trials “‘

A trial is counted as a verified recovery only if all of the following are true: the failure is detected, the root cause is classified, a remediation action is available, the remediation is policy-approved, the remediation executes successfully, and post-recovery validation passes.

Runtime measures the operational cost of the self-healing cycle. Runtime is tracked separately from accuracy and recovery because a system can be accurate but operationally impractical if the detection-to-validation cycle is too slow.

G. Recovery Success Definition

The paper uses a strict recovery success definition. Recovery is successful only when the system restores the pipeline to a validated healthy state after an approved remediation.

The following are not counted as successful recovery: detecting a failure without remediation, classifying a root cause without remediation, selecting a remediation that policy rejects, executing a remediation without post-recovery validation, executing a remediation that validation fails, escalating a failure to a human operator, rolling back an unsafe remediation, or suppressing an alert or bypassing validation.

This definition prevents the experiment from overstating self-healing performance. It also makes the verified recovery rate more credible than a larger but weaker automation-attempt rate.

H. Reproducibility Artifacts

The repository is organized to support reproducibility. The experiment artifacts include implementation code, experiment configurations, raw trial records, derived result summaries, policies, remediation registry artifacts, telemetry records, incident evidence, scripts, figures, and documentation.

The figure-generation workflow uses three primary derived inputs: combined raw experiment results, scenario-level summaries, and a classification confusion matrix. The generated publication figures include detection and root-cause classification accuracy by scenario, verified recovery outcomes by failure scenario, self-healing cycle runtime distribution, and root-cause classification confusion matrix.

The analysis is intended to remain reproducible from preserved trial-level results. Final claims should be traceable to raw or derived experiment outputs.

I. Assumptions and Scope

The current methodology depends on several assumptions. The synthetic failure taxonomy is assumed to represent meaningful data-pipeline failure classes. Injected failures are assumed to produce observable signals that the detector can evaluate. Expected root-cause labels are known for injected failure trials. Recovery actions are available only for a subset of failure types. Policy constraints may intentionally block remediation. Post-recovery validation is required before recovery is counted as successful.

The scope is intentionally limited. The current experiment does not prove that the framework will achieve the same detection, classification, or recovery performance on real enterprise incidents. It demonstrates feasibility and reproducibility under controlled synthetic conditions.

J. Implementation Summary

The current implementation represents the self-healing lifecycle as a deterministic experimental control plane. Trial configurations define the experiment domain, random seed, injected failure type, severity, and expected root cause. Pipeline execution produces observed records and artifact metadata, including input and output record counts, quarantined record counts, size observations, checksum status, and runtime measurements.

Failure detection is evaluated through trial-level detection outcomes rather than through an unconstrained production anomaly detector. Root-cause classification maps detected failures to the predefined taxonomy and records whether the predicted root cause matches the injected condition. Remediation is represented through a registry-style action field, execution status, human-approval flag, cycle status, and post-recovery verification outcome. Policy constraints are reflected in whether a remediation is allowed, whether human approval is required, whether the remediation is executed, and whether recovery is verified after execution.

This implementation is intentionally conservative. It is designed to test whether the staged evaluation model is reproducible and measurable, not to prove that the current detector or classifier generalizes to arbitrary production incidents. Future implementations can replace deterministic detection and taxonomy-based classification with probabilistic, learning-based, or hybrid methods while preserving the same evaluation structure.

IV. Results

A. Experiment Summary

The experimental evaluation produced 780 total trial records spanning injected failures, healthy controls, and boundary-control scenarios. Of these, 690 trials corresponded to injected data-pipeline failure conditions, 60 trials represented healthy-control executions, and 30 trials represented boundary-control conditions. The experiment design therefore evaluates both positive failure cases and non-failure controls rather than measuring the detector only on failure-positive examples.

Artifact availability: The repository includes the raw outputs, derived summaries, confusion-matrix data, and publication-ready figures required to reproduce the reported results; Table IV lists the corresponding paths.

Artifact availability: The repository includes the raw outputs, derived summaries, confusion-matrix data, and publication-ready figures required to reproduce the reported results; Table IV lists the corresponding paths.

Artifact availability: The repository includes the raw outputs, derived summaries, confusion-matrix data, and publication-ready figures required to reproduce the reported results; Table IV lists the corresponding paths.

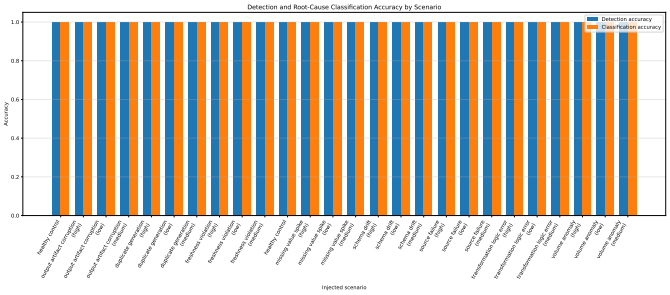

Fig. 1. Detection and root-cause classification accuracy by injected scenario.

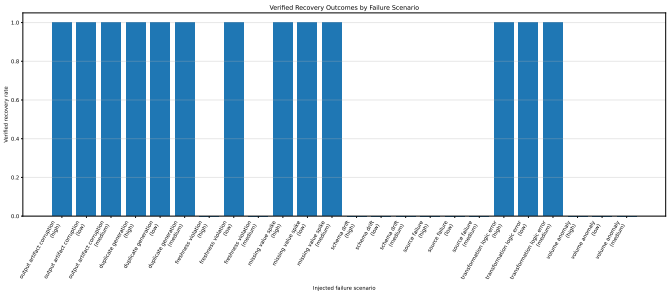

Fig. 2. Verified recovery outcomes by injected failure scenario.

V. Discussion

A. Detection and Diagnosis Are Necessary but Insufficient

The experimental results support the view that policy-constrained self-healing should be understood as a staged reliability-control process. Detection, diagnosis, remediation selection, execution, validation, rollback, and escalation are distinct responsibilities. The 100% detection and classification results demonstrate internal consistency under the current synthetic taxonomy, but they should not be interpreted as production generalization.

B. Policy Constraints Are a Reliability Feature

The verified recovery rate reflects the system design choice to enforce policy approval and post-recovery validation. In enterprise data pipelines, an automated remediation that corrupts downstream data, violates governance rules, or masks a serious source-system problem is worse than controlled escalation. Counting only verified recovery makes the recovery metric more conservative and more useful than a metric that counts every remediation attempt as success.

C. Escalation Is a Valid Safety Outcome

A mature self-healing system should not attempt to repair every detected failure. Some failures are ambiguous,

TABLE IV
ARTIFACT AVAILABILITY FOR REPRODUCIBILITY.

Artifact	Path	Status
Raw combined results	`experiments/raw_results/combined_experiment_results.csv`	Found
Scenario summary	`experiments/derived_results/scenario_summary.csv`	Found
Classification confusion matrix	`experiments/derived_results/classification_confusion_matrix.csv`	Found
Figure 1 PDF	`figures/figure_1_accuracy_by_scenario.pdf`	Found
Figure 2 PDF	`figures/figure_2_recovery_by_scenario.pdf`	Found
Figure 3 PDF	`figures/figure_3_runtime_distribution.pdf`	Found
Figure 4 PDF	`figures/figure_4_classification_confusion_matrix.pdf`	Found

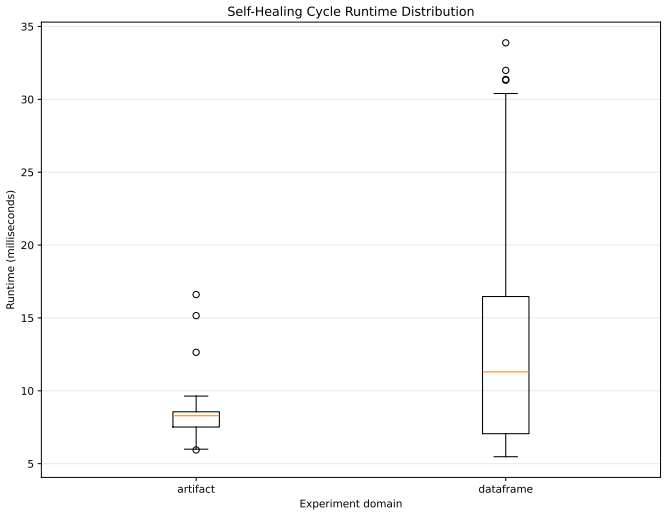

Fig. 3. Self-healing cycle runtime distribution.

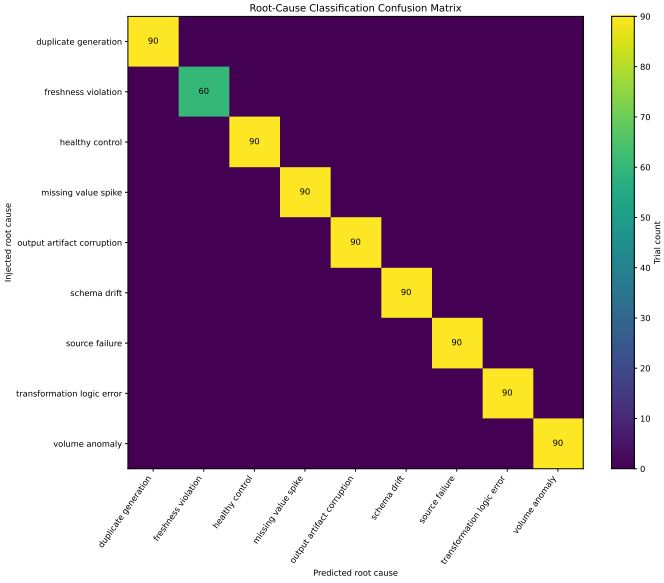

Fig. 4. Root-cause classification confusion matrix.

governance-sensitive, unsupported by the remediation registry, or unsafe to repair automatically. In those cases, escalation is a valid reliability outcome. The system preserves evidence and avoids unsafe action rather than forcing automation.

This distinction is especially important for regulated or high-dependence data environments. Data platforms used for insurance, finance, healthcare, compliance reporting, or operational analytics often require traceability and defensible control points. In such environments, self-healing cannot be judged purely by speed or automation rate. It must also preserve evidence, enforce guardrails, and make failure-handling decisions auditable.

D. Runtime Interpretation

The runtime results show that the evaluated self-healing cycle executed within millisecond-scale runtimes in the current synthetic environment. However, these results should be interpreted cautiously. The experiment does not prove that equivalent runtimes would hold for large production datasets, distributed infrastructure, external systems, or human-approval workflows. Runtime evidence is useful for feasibility, but production latency must be evaluated separately under realistic operational conditions.

E. Design Implications

The results support a narrower but stronger contribution: a reproducible experimental framework for evaluating data-pipeline self-healing as a policy-constrained control plane. The contribution is not that every pipeline failure can be repaired automatically. The contribution is that failure detection, root-cause classification, safe remediation selection, validation, and escalation can be represented as measurable stages with separate outcomes.

Future self-healing data systems should separate detection, diagnosis, and recovery metrics; preserve failed recovery attempts; enforce policy constraints; validate recovered state; and maintain incident evidence. These design principles make self-healing safer and more auditable than unrestricted remediation automation.

VI. THREATS TO VALIDITY

A. Internal Validity

The experiment uses controlled failure injection. This improves reproducibility but may simplify the relationship between injected failures and observable telemetry. A detector may perform better in this environment than in a production system where failures are noisy, overlapping, delayed, or partially observable.

B. Construct Validity

The metrics separate detection accuracy, root-cause classification accuracy, and verified recovery rate. This is appropriate for the paper's research objective, but these metrics do not capture every operational concern. For example, they do not fully measure operator trust, downstream business impact, long-term model drift, or the cost of false escalation.

C. External Validity

The results should be generalized carefully. The experiment currently evaluates a predefined taxonomy of failure conditions. Real-world data platforms may include infrastructure failures, vendor API instability, semantic data-quality defects, access-control issues, cross-system contract changes, and cascading downstream failures that are not fully represented in the current test set.

D. Runtime Validity

Runtime results were measured in the current synthetic experiment environment. These runtime measurements provide evidence about the local experimental setup, but they do not establish production latency under distributed workloads, large-scale datasets, external storage systems, network calls, or manual approval steps. Runtime claims should therefore remain limited to the preserved experimental records.

E. Conclusion Validity

The experiment provides evidence that the framework works under the current controlled setup. It does not prove superiority over mature observability products or enterprise incident-management workflows. Future experiments should compare the policy-constrained framework against alert-only monitoring and static rule-based automation using identical failure scenarios and operational metrics.

F. Baseline Comparison Scope

Claims involving baseline comparison are limited to experiment domains represented in the preserved trial records. The current preserved domains are `artifact` and `dataframe`; they do not constitute alert-only or static-rule baseline domains. Broader comparison against alert-only or static-rule systems remains future work unless evaluated under identical scenarios and metrics.

G. Synthetic Data and Failure Taxonomy Bias

Because the failure taxonomy is predefined and synthetic, the evaluation may favor the implemented detector and classifier. The 100% detection and classification results should therefore be interpreted as evidence that the framework is internally coherent under the current experiment configuration, not as evidence that the same performance would hold for independently constructed failures or production incidents. Future work should evaluate the framework on real incident records, independently generated failure scenarios, or larger benchmark suites.

VII. Conclusion and Future Work

This paper presented a reliability-aware self-healing control plane for data pipelines. The framework models self-healing as a staged process involving failure detection, root-cause classification, policy-constrained remediation, execution, post-recovery validation, rollback or escalation, and evidence preservation. The experimental evaluation used a reproducible synthetic failure-injection setup with 780 total trials, including injected failures, healthy controls, and boundary controls.

Under the current controlled experiment configuration, the framework achieved 100% failure detection accuracy and 100% root-cause classification accuracy, while verified recovery was achieved in 52.17% of injected failure trials. Runtime was measured across 780 records, with a mean of 12.609 ms, median of 10.609 ms, and p95 of 26.666 ms. These results support feasibility under controlled conditions, while also showing that detection and diagnosis should not be treated as equivalent to verified recovery.

The results support the view that self-healing data pipelines should be evaluated as governed reliability-control systems rather than unrestricted automation scripts. Future work should evaluate the framework on real production incidents, expand the failure taxonomy, compare against alert-only and static-rule baselines under identical scenarios, incorporate probabilistic or learning-based root-cause models, and study the operational tradeoffs between automation, escalation, safety, and recovery latency.

References

[1] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers," 2023.

[2] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, 2015.

[3] D. Xin, H. Miao, A. Parameswaran, and N. Polyzotis, "Production machine learning pipelines: Empirical analysis and optimization opportunities," 2021.

[4] Apache Software Foundation, "Dags — airflow documentation," https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html, 2026, accessed: 2026-06-15.

[5] Great Expectations, "Try gx core," https://docs.greatexpectations.io/docs/core/introduction/try_gx/, 2026, accessed: 2026-06-15.

[6] dbt Labs, "Add data tests to your dag," https://docs.getdbt.com/docs/build/data-tests, 2026, accessed: 2026-06-15.

[7] Open Policy Agent, "Open policy agent documentation," https://www.openpolicyagent.org/docs, 2026, accessed: 2026-06-15.

[8] Google, "Monitoring distributed systems," https://sre.google/sre-book/monitoring-distributed-systems/, 2026, accessed: 2026-06-15.

[9] D. Tu, Y. He, W. Cui, S. Ge, H. Zhang, H. Shi, D. Zhang, and S. Chaudhuri, "Auto-validate by-history: Auto-program data quality constraints to validate recurring data pipelines," 2023.

[10] T. Rehberger, T. Hütter, L. Ehrlinger, and W. Wöß, "Evaluating data quality tools: Measurement capabilities and llm integration," 2026.

[11] J. Yasmin, J. Wang, Y. Tian, and B. Adams, "An empirical study of developers' challenges in implementing workflows as code: A case study on apache airflow," 2024.

[12] P. Notaro, J. Cardoso, and M. Gerndt, "A systematic mapping study in aiops," 2020.

[13] A. Saha and S. C. H. Hoi, "Mining root cause knowledge from cloud service incident investigations for aiops," 2022.

[14] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.